

Chapter 14 Minimum Spanning Trees

Minimum Spanning Trees (MSTs)

◆ Prompt

What is the Minimum Spanning Tree (MST) problem?

✓ Answer

- A **spanning tree** of a connected, undirected graph G is a tree that contains **all vertices** of G and a subset of edges such that there are **no cycles**.
- A **Minimum Spanning Tree (MST)** is a spanning tree whose **total edge weight** is as **small as possible**.
- Formally, given an undirected, weighted graph $G = (V, E)$ with weight function $w(u, v)$, we want a tree $T \subseteq E$ such that:

$$w(T) = \sum_{(u,v) \in T} w(u,v) \text{ is minimized.}$$

Use case example:

- Connecting all computers in an office with the **least total cable length**.

◆ Prompt

Why do we care about MSTs?

✓ Answer

- MSTs minimize the **total cost** of connecting a network.
- Applications include:
 1. **Network design** (cables, fiber optics, roads, pipelines)
 2. **Clustering algorithms** in data analysis
 3. **Approximation algorithms** for NP-hard problems like the traveling salesman

Key property of MSTs

◆ Prompt

What is the key property of MSTs that enables greedy algorithms?

✓ Answer

- **Greedy property / Cut property:**
 - For any subset of vertices $S \subset V$, the **lightest edge** connecting SS to $V \setminus S$ must be part of **some MST**.
- This property allows algorithms to **incrementally build MSTs** by always picking the smallest available edge that doesn't form a cycle.

◆ Prompt

What are the two main algorithms to find an MST?

✓ Answer

Prim-Jarník Algorithm (Prim's Algorithm)

- Starts with a single vertex.
- Grows the MST by **adding the cheapest edge** connecting a vertex in the tree to a vertex outside.
- Similar to **Dijkstra's algorithm** for shortest paths.

Kruskal's Algorithm

- Starts with all vertices as separate components.
- Sorts edges by weight and **adds edges in increasing weight**, skipping edges that form cycles.
- Grows the MST in **clusters**, eventually connecting all components.

Comparison of Prim vs Kruskal

Property	Prim Algorithm	Kruskal Algorithm
Start point	Single vertex	Edge with smallest weight
Approach	Tree grows vertex by vertex	Tree grows edge by edge
Data structures	Priority queue (heap)	Union-Find (Disjoint sets)
Best for	Dense graphs	Sparse graphs

Example Graph and MST

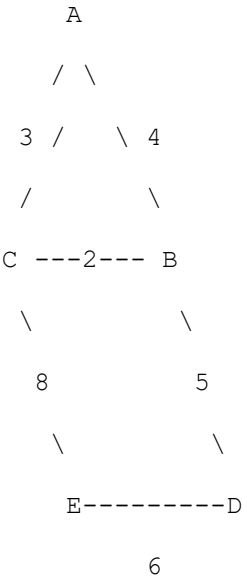
Graph:

Edge	Weight
A-B	4
A-C	3
B-C	2
B-D	5
C-D	7
C-E	8
D-E	6

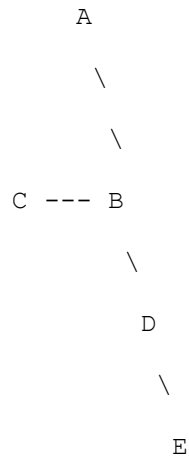
MST by Prim/Kruskal:

- **Edges in MST:** B-C (2), A-C (3), B-D (5), D-E (6)
- **Total weight:** $2 + 3 + 5 + 6 = 16$

Diagram:



MST edges highlighted:



Steps of Prim's Algorithm (Example)

1. Start at vertex A
2. Add **smallest edge from A** $\rightarrow$ A-C (3)
3. Add **smallest edge from {A,C}** $\rightarrow$ C-B (2)
4. Add **smallest edge from {A,C,B}** $\rightarrow$ B-D (5)
5. Add **smallest edge from {A,C,B,D}** $\rightarrow$ D-E (6)
6. MST complete

Steps of Kruskal's Algorithm (Example)

1. Sort edges by weight: B-C (2), A-C (3), A-B (4), B-D (5), D-E (6), C-D (7), C-E (8)
2. Pick edges **in increasing weight** while avoiding cycles:
 - B-C (2) $\rightarrow$ add
 - A-C (3) $\rightarrow$ add
 - A-B (4) $\rightarrow$ forms cycle $\rightarrow$ skip
 - B-D (5) $\rightarrow$ add
 - D-E (6) $\rightarrow$ add
 - Remaining edges $\rightarrow$ skip (would form cycles)

MST complete

Prim-Jarník Algorithm (Prim's Algorithm)

◆ Prompt

What is the Prim-Jarník algorithm for MSTs?

✓ Answer

- The **Prim-Jarník algorithm** builds a **minimum spanning tree (MST)** incrementally, starting from a single vertex (root).
- It maintains a “**cloud**” of vertices **C** that are already included in the MST.
- At each step, it **adds the minimum-weight edge** that connects a vertex inside the cloud **C** to a vertex outside **CC**.
- This continues until **all vertices are included**, forming a complete MST.

Key idea: Always pick the **cheapest edge connecting the MST to a new vertex**.

◆ Prompt

How does Prim's Algorithm efficiently choose the next vertex?

✓ Answer

- Maintain a **label** $D[v]$ **D[v]** for each vertex vv outside the cloud **CC**, storing the **weight of the minimum edge connecting v to C**.
- Use a **priority queue** keyed by $D[v]$ **D[v]** to select the vertex vv with the **smallest connecting edge**.
- When a new vertex uu is added, **update** $D[v]$ **D[v]** for all neighbors vv outside **CC** if a cheaper edge through uu exists.

This is similar to **Dijkstra's algorithm**, except we only care about the **single edge weight**, not the full path.

◆ Prompt

What is the pseudocode of Prim's Algorithm?

✓ Answer

Algorithm Prim-Jarník:

Input: Connected, undirected, weighted graph $G = (V, E)$

Output: MST T

1. Pick any vertex s
2. $D[s] = 0$
3. For each vertex $v \neq s$, $D[v] = \infty$
4. Initialize MST $T = \emptyset$
5. Initialize priority queue Q with entries $(D[v], (v, \text{None}))$ for all v
6. While Q is not empty:
 - $(u, e) = Q.\text{remove_min}()$ # vertex u with min $D[u]$
 - Add u to MST using edge e
 - For each edge $e = (u, v)$ with v in Q :
 - if $w(u, v) < D[v]$:
 - $D[v] = w(u, v)$
 - Update key of v in Q to $D[v]$
 - Update value of v in Q to (v, e)
7. Return T

Step-by-Step Example

Graph:

Edge	Weight
A-B	4
A-C	3
B-C	2
B-D	5
C-D	7
C-E	8
D-E	6

Steps (Starting at A):

Start **cloud** $C = \{A\}$, priority queue labels:

- $D[A] = 0, D[B] = \infty, D[C] = \infty, D[D] = \infty, D[E] = \infty$

Pick min $D[v] = A \rightarrow$ add A to MST

Update neighbors of A:

- Edge A-B (4) $\rightarrow D[B] = 4$
- Edge A-C (3) $\rightarrow D[C] = 3$

Pick min $D[v] = C \rightarrow$ add C to MST (edge A-C)

Update neighbors of C:

- Edge C-B (2) $\rightarrow D[B] = 2$ (smaller than previous 4)
- Edge C-D (7) $\rightarrow D[D] = 7$
- Edge C-E (8) $\rightarrow D[E] = 8$

Pick min $D[v] = B \rightarrow$ add B to MST (edge C-B)

Update neighbors of B:

- Edge B-D (5) $\rightarrow D[D] = 5$ (smaller than previous 7)

Pick min $D[v] = D \rightarrow$ add D to MST (edge B-D)

Update neighbors of D:

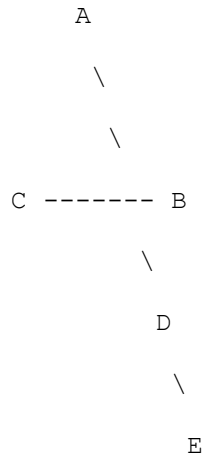
- Edge D-E (6) $\rightarrow D[E] = 6$ (smaller than previous 8)

Pick min $D[v] = E \rightarrow$ add E to MST (edge D-E)

MST complete: **edges** = {**A-C, C-B, B-D, D-E**}, total weight = 16

Visual Diagram

Graph with MST highlighted:



Edge weights in MST:

- A-C = 3
- C-B = 2
- B-D = 5
- D-E = 6

Other edges (not in MST):

- A-B = 4, C-D = 7, C-E = 8

Time Complexity

◆ Prompt

What is the time complexity of Prim-Jarník Algorithm?

✓ Answer

- Using a **binary heap** / **priority queue**:
 - $O((n + m) \log n)$, where $n = |V|$, $m = |E|$
- Using a **Fibonacci heap** (advanced):
 - $O(m + n \log n)$

Analyzing and Illustrating the Prim-Jarník Algorithm

◆ Prompt

How efficient is the Prim-Jarník algorithm, and what factors affect its running time?

✓ Answer

- The running time depends on the priority queue implementation:

Operation	Count	Cost per Operation	Total Cost
Insert all n vertices into Q	n	$O(\log n)$	$O(n \log n)$
Extract-min n times	n	$O(\log n)$	$O(n \log n)$
Update edge weights (key decrease)	up to m	$O(\log n)$	$O(m \log n)$

- **Total running time:**
 - $O((n + m) \log n)$
 - For a **connected graph**, $m \geq n-1$, so it simplifies to **$O(m \log n)$** .
- Using a **simple unsorted list** for Q :
 - Extract-min = $O(n) \rightarrow$ total time = $O(n^2)$
 - Less efficient for large graphs.

◆ Prompt

Step-by-step process of Prim-Jarník MST growth.

✓ Answer

1. **Start at a root vertex**, e.g., PVD.
2. **Initialize the cloud** $C = \{PVD\}$ and label $D[v] = \infty$ for all other vertices.
3. **Pick the smallest edge connecting C to outside vertices** using priority queue Q .
4. **Add the vertex at the other end of that edge to C .**
5. **Update $D[v]$ for neighbors outside C** if a smaller connecting edge is found.
6. Repeat steps 3–5 until all vertices are included.

Illustration of Prim-Jarník MST (Airports Example)

We can illustrate the MST step by step based on the figures you referenced (ORD, SFO, LAX, etc.):

Graph vertices (airports):

- PVD, ORD, SFO, LAX, DFW, BOS, JFK, BWI, MIA

Step-by-step MST formation (edge weights in parentheses):

Step	Added Vertex	Edge Added	MST So Far
1	PVD	—	PVD
2	BOS	PVD-BOS (144)	PVD-BOS
3	JFK	BOS-JFK (184)	PVD-BOS-JFK
4	LAX	JFK-LAX (337)	...
5	DFW	LAX-DFW (802)	...
6	SFO	LAX-SFO (1464)	...
7	ORD	DFW-ORD (1846)	...
8	BWI	BOS-BWI (946)	...
9	MIA	BWI-MIA (1090)	...

Total MST edges: 8 edges for 9 vertices, as expected.

Python Implementation of Prim-Jarník Algorithm

Understanding the Python Code

The function `MSTPrimJarnik(g)` computes the minimum spanning tree (MST) of a weighted, undirected graph `g`.

Here's the breakdown:

```
def MSTPrimJarnik(g):    """Compute a minimum spanning tree of weighted
graph g.

    Returns a list of edges that comprise the MST (in arbitrary order).

    """
    d = {}               # d[v] stores minimum weight to connect v to
the MST

    tree = []            # list of edges in the MST

    pq = AdaptableHeapPriorityQueue() # priority queue of vertices by d[v]

    pqlocator = {}       # maps vertex to its position in pq
```

- `d[v]`: The smallest weight edge that connects vertex `v` to the growing MST.
- `tree`: Collects edges that are added to the MST.
- `pq`: Adaptable priority queue (like a heap) to efficiently get the vertex with the smallest `d[v]`.
- `pqlocator`: Allows efficient update of a vertex's priority in the heap.

Step: Initialize Priority Queue

```
for v in g.vertices():
    if len(d) == 0:
        d[v] = 0          # first vertex chosen as root of MST
    else:
        d[v] = float('inf') # others initially infinite
    pqlocator[v] = pq.add(d[v], (v, None)) # add vertex to priority queue
```

- The **first vertex** has `d[v] = 0` → will be chosen first.
- All **other vertices** have `d[v] = ∞`.
- Each vertex is added to the heap with its current weight.

Step: Main Loop – Build MST

```
while not pq.isempty():
    key, value = pq.removemin()

    u, edge = value # unpack vertex and the edge that discovered it
    del pqlocator[u] # remove vertex u from PQ (now in MST)

    if edge is not None:
        tree.append(edge) # add edge to MST
```

- At each iteration, we pick the **vertex with the minimum connection weight**.
- `edge` is the edge used to bring `u` into the MST.
- If `edge` is not `None`, we add it to the MST.

Step: Update Neighbors

```
for link in g.incident_edges(u):
```

```

v = link.opposite(u)

if v in pqlocator:    # only consider vertices not yet in MST

    wgt = link.element()    # weight of edge

    if wgt < d[v]:        # smaller weight found?

        d[v] = wgt        # update best connection weight

        pq.update(pqlocator[v], d[v], (v, link))    # update priority
in heap

```

- For every **neighbor v of u not in MST**, we check if connecting via u gives a **smaller edge weight** than the current $d[v]$.
- If yes, **update $d[v]$** and update its position in the heap.

Step: Return the MST

```
return tree    # list of edges in the MST
```

- After all vertices are included in the MST, return the **list of edges** forming the MST.

Key Concepts Highlighted

Concept	Explanation
$d[v]$	Minimum edge weight connecting v to MST
pq	Priority queue to efficiently select vertex with smallest $d[v]$
pqlocator	Allows updating vertex priorities in the heap
tree	MST edges collected during execution
Complexity $O((n + m) \log n)$ using heap-based priority queue	

Step-by-Step Example (Illustration)

Let's take a **small weighted graph**:

Vertices: A, B, C, D

Edges with weights:

A-B: 1

A-C: 3

B-C: 3

B-D: 6

C-D: 2

Step 1: Initialize

- Choose **A as root** $\rightarrow d[A] = 0$, others ∞
- $PQ = [(A,0), (B,\infty), (C,\infty), (D,\infty)]$

Step 2: Pick min vertex from PQ

- Remove **A ($d[A]=0$)** $\rightarrow$ add edges from A to neighbors:
 - A-B: 1 $\rightarrow d[B]$ updated to 1
 - A-C: 3 $\rightarrow d[C]$ updated to 3
- $PQ = [(B,1), (C,3), (D,\infty)]$

Step 3: Pick min vertex from PQ

- Remove **B ($d[B]=1$)** $\rightarrow$ add edges from B to neighbors:
 - B-C: 3 $\rightarrow d[C] = 3$ (no change)
 - B-D: 6 $\rightarrow d[D] = 6$
- $PQ = [(C,3), (D,6)]$
- MST edges = [A-B]

Step 4: Pick min vertex from PQ

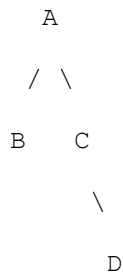
- Remove **C ($d[C]=3$)** $\rightarrow$ add edges from C:
 - C-D: 2 $\rightarrow d[D]$ updated to 2
- $PQ = [(D,2)]$
- MST edges = [A-B, A-C]

Step 5: Pick min vertex from PQ

- Remove **D ($d[D]=2$)** $\rightarrow$ no neighbors outside MST
- MST edges = [A-B, A-C, C-D]

✓ **MST Total Weight:** $1 + 3 + 2 = 6$

Visual Diagram of MST Growth



- **Edge weights:** A-B=1, A-C=3, C-D=2
- **Edges in MST:** [A-B, A-C, C-D]

Step-by-step additions match the priority queue decisions:

1. Pick A → add A-B, A-C
2. Pick B → add B-D? (weight 6 > 2) → ignore
3. Pick C → add C-D (smaller weight 2)
4. Pick D → done

Kruskal's Algorithm (Minimum Spanning Tree)

◆ **PROMPT 1: What problem does Kruskal's algorithm solve?**

✓ **Answer:**

Kruskal's algorithm finds a **Minimum Spanning Tree (MST)** of a **connected, undirected, weighted graph**.

A **minimum spanning tree**:

- Connects **all vertices**
- Uses exactly **$n - 1$ edges**
- Has **minimum possible total edge weight**
- Contains **no cycles**

◆ PROMPT 2: What is the main idea behind Kruskal's algorithm?

✓ Answer:

Kruskal's algorithm:

- Starts with **each vertex as its own cluster**
- Sorts **all edges by increasing weight**
- Repeatedly **adds the smallest edge** that:
 - Connects **two different clusters**
 - Does **not create a cycle**
- Stops when **$n - 1$ edges** have been added

Instead of growing **one tree** (like Prim), it grows **many small trees (a forest)** and merges them.

◆ PROMPT 3: How is Kruskal's algorithm different from Prim–Jarník?

Prim–Jarník	Kruskal
Grows one tree	Grows a forest of trees
Starts from a vertex	Starts from edges
Uses vertex priority queue	Uses edge priority queue
Similar to Dijkstra	Uses union of clusters

◆ PROMPT 4: What are “clusters” in Kruskal's algorithm?

✓ Answer:

A **cluster** is a connected component formed so far.

- Initially: each vertex is its **own cluster**
- When an edge connects two clusters → clusters are **merged**
- If an edge connects vertices in the **same cluster** → rejected (cycle)

Clusters are usually implemented using **Disjoint Set / Union-Find**.

◆ PROMPT 5: What is the pseudocode logic of Kruskal's algorithm?

✓ Answer (explained version):

1. Make each vertex its **own cluster**
2. Put **all edges** into a priority queue (sorted by weight)
3. Initialize MST $T = \emptyset$
4. While T has fewer than $n - 1$ edges:
 - Remove smallest edge (u, v)
 - If u and v are in **different clusters**:
 - Add edge to T
 - Merge the two clusters
 - Else:
 - Discard the edge
1. Output T

◆ PROMPT 6: Why does Kruskal's algorithm work (correctness)?

✓ Answer:

Kruskal's algorithm is **greedy**.

It relies on the **Cut Property**:

For any partition (cut) of the vertices, the **minimum-weight edge crossing the cut** is always part of some MST.

At every step:

- Clusters define a cut
- The smallest edge connecting two clusters is **safe**
- Therefore, every added edge is guaranteed to be part of the MST

◆ PROMPT 7: Step-by-step example

Example Graph:

Vertices: A, B, C, D

Edges (weight):

A-B : 1

C-D : 2

B-C : 3

A-C : 4

B-D : 5

Step 1: Sort edges by weight

A-B (1)

C-D (2)

B-C (3)

A-C (4)

B-D (5)

Step 2: Initialize clusters

{A} {B} {C} {D}

MST = $\emptyset$

Step 3: Process edges

✓ *Edge A-B (1)*

- A and B are in different clusters
- Add edge
- Merge {A} and {B}

Clusters: {A,B} {C} {D}

MST: A-B

✓ *Edge C-D (2)*

- Different clusters
- Add edge
- Merge {C} and {D}

Clusters: {A,B} {C,D}

MST: A-B, C-D

✓ Edge B-C (3)

- {A,B} and {C,D} are different
- Add edge
- Merge clusters

Clusters: {A,B,C,D}

MST: A-B, C-D, B-C

✓ MST complete ($n-1 = 3$ edges)

◆ PROMPT 8: What edges are rejected and why?

✓ Answer:

Edges that connect vertices **already in the same cluster** are rejected because:

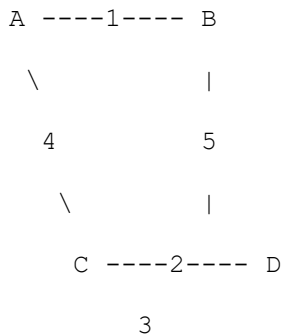
- They create a **cycle**
- MST must be acyclic

Example:

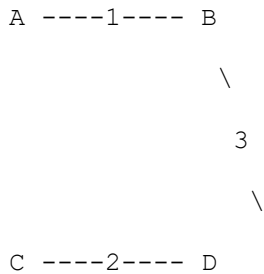
- Edge A-C (4) is rejected after clusters are merged

◆ PROMPT 9: ASCII Diagram (Visual Explanation)

◆ Original Graph



◆ MST Produced by Kruskal



Edges chosen:

- A–B (1)
- C–D (2)
- B–C (3)

◆ PROMPT 10: Time Complexity of Kruskal's Algorithm

✓ Answer:

Operation	Cost
Sorting edges	$O(m \log m)$
Union-Find operations	$O(m \alpha(n)) \approx O(m)$
Total	$O(m \log m)$

Where:

- n = number of vertices
- m = number of edges
- $\alpha(n)$ = inverse Ackermann function (very small)

◆ PROMPT 11: When should we use Kruskal's algorithm?

✓ Answer:

Kruskal is best when:

- Graph is **sparse**
- Edges are already sorted or easy to sort
- Union-Find is efficient

Prim is often better for **dense graphs**.

Python Implementation of Kruskal's Algorithm

(Code Fragment 14.18)

◆ PROMPT 1: What does this function do?

✓ Answer:

`MSTKruskal(g)` computes the **Minimum Spanning Tree (MST)** of a **connected, undirected, weighted graph** using **Kruskal's algorithm**.

- Input: graph g
- Output: a **list of edges** forming the MST
- Edges are returned in **nondecreasing order of weight**

◆ PROMPT 2: What assumptions does this implementation make?

✓ Answer:

This implementation assumes:

- The graph is **connected**
- The graph is **undirected**
- The **edge element is the weight**
- A **Partition (Union–Find)** data structure is available
- A **HeapPriorityQueue** is used to sort edges by weight

◆ PROMPT 3: What data structures are used and why?

✓ Answer:

Structure	Purpose
<code>tree</code>	Stores MST edges
<code>HeapPriorityQueue</code>	Sorts edges by increasing weight
<code>Partition</code>	Manages clusters (Union–Find)
<code>position</code>	Maps each vertex to its partition entry

◆ PROMPT 4: What is the role of the `Partition` class?

✓ Answer:

The `Partition` class:

- Keeps track of **connected components (clusters)**
- Supports:
 - `makegroup(v)` → create new cluster
 - `find(x)` → find cluster representative
 - `union(a,b)` → merge clusters

This prevents **cycles** by ensuring we only connect **different clusters**.

◆ PROMPT 5: Step-by-step explanation of the code

◆ Code (for reference)

```
def MSTKruskal(g):
```

➡ Define function to compute MST

```
tree = []
```

➡ Empty list to store MST edges

```
pq = HeapPriorityQueue()
```

➡ Priority queue to store edges sorted by weight

```
forest = Partition()
```

➡ Union-Find structure to manage clusters

```
position = {}
```

➡ Map each vertex to its partition entry

```
for v in g.vertices():
```

```
    position[v] = forest.makegroup(v)
```

➡ Each vertex starts as **its own cluster**

```
for e in g.edges():
```

```
    pq.add(e.element(), e)
```

➡ Insert all edges into priority queue

➡ Key = weight, Value = edge

```
size = g.vertexcount()
```

➡ Number of vertices n

```
while len(tree) != size - 1 and not pq.isempty():
```

➡ Continue until:

- MST has $n - 1$ edges
- OR no edges remain

```
weight, edge = pq.removemin()
```

➡ Remove smallest-weight edge

```
u, v = edge.endpoints()
```

➡ Get edge endpoints

```
a = forest.find(position[u])
```

```
b = forest.find(position[v])
```

➡ Find clusters of u and v

```
if a != b:
```

➡ Only proceed if they are in **different clusters**

```
tree.append(edge)
```

```
forest.union(a, b)
```

➡ Add edge to MST

➡ Merge clusters

```
return tree
```

➡ Return MST edges

◆ PROMPT 6: Why does this implementation avoid cycles?

✓ Answer:

A cycle occurs **only if both endpoints belong to the same cluster**.

This line prevents cycles:

```
if a != b:
```

If `a == b`, the edge is discarded.

◆ PROMPT 7: Why are edges returned in sorted order?

✓ Answer:

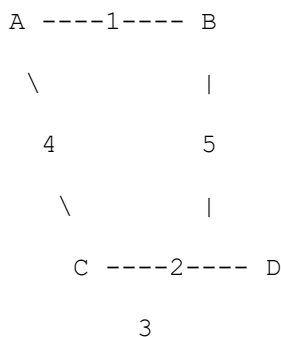
Edges are:

- Inserted into a **priority queue**
- Removed using `removeMin()`

Therefore, edges are processed and added in **nondecreasing weight order**.

◆ PROMPT 8: Visual Illustration (ASCII Diagram)

◆ Input Graph



◆ Sorted Edges

A-B (1)

C-D (2)

B-C (3)

A-C (4)

B-D (5)

◆ MST Construction (Kruskal)

Step 1: A-B (1) ✓

Clusters: {A,B} {C} {D}

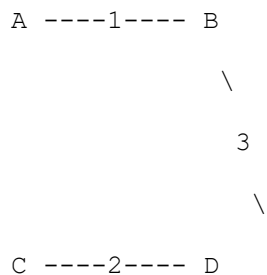
Step 2: C-D (2) ✓

Clusters: {A,B} {C,D}

Step 3: B-C (3) ✓

Clusters: {A,B,C,D}

◆ Final MST



◆ PROMPT 9: Time Complexity of this implementation

✓ Answer:

Step	Complexity
Insert edges into PQ	$O(m)$
Sort edges (heap)	$O(m \log m)$
Union-Find ops	$O(m \alpha(n))$
Total	$O(m \log m)$

($\alpha(n)$ is very small — treated as constant)